ChatGPT

Vytvoř kompletní scaffold webové aplikace PhysioBook, rezervační systém pro fyzioterapeuty s online platbami. Jedná se o demo projekt. Aplikace umožňuje klientům rezervovat si termíny u fyzioterapeutů a platit online přes Stripe, avšak teď bude za úkol vytvořit pouze scaffold, article entitu a základní CRUD na articles.

Tech Stack

- Backend: ASP.NET Core 8.0 (Minimal API)
- Frontend: React 18 + Vite + TypeScript + TailwindCSS
- Databáze: PostgreSQL 15 + Entity Framework Core 8 (Npgsql)
- Autentizace: ASP.NET Identity + JWT Bearer tokeny
- Mediator/CQRS: Cortex.Mediator (NuGet `Cortex.Mediator`) — verze 1.6.1+
- Validace: FluentValidation + pipeline behavior
- Testy: xUnit + FluentAssertions + Moq
- Kontejnerizace: Docker + docker-compose (vše v kontejnerech — backend, frontend, PostgreSQL)

Architektura: Vertical Slice + CQRS

Projekt používá Vertical Slice Architecture s CQRS patternem. Každá operace (command/query) žije ve vlastním adresáři se 7 soubory.

Adresářová struktura backendu

```
PhysioBook/
├── Auth/
│   ├── AppPolicies.cs            # Authorization policies (Visitor, Admin (Therapist))
│   ├── AppRoles.cs               # Role constants (Admin (Therapist))
│   └── JwtAuthSettings.cs        # JWT + Identity konfigurace
├── Configurations/
│   ├── ServiceRegistrations.cs   # DI registrace všech služeb
│   ├── AppConfiguration.cs       # Middleware pipeline
│   ├── EndpointRegistration.cs   # Centrální registrace VŠECH endpointů
│   └── DatabaseConfiguration.cs  # EF migrace + seed
├── Data/
│   ├── PhysioBookContext.cs      # DbContext (IDbContextFactory pattern)
│   ├── Entities/                 # Doménové entity + IEntityTypeConfiguration v jednom souboru
│   ├── Enums/                    # Business enumerace
│   ├── Migrations/
│   └── Shared/
│       ├── PagedList.cs          # PagedList<T> pro stránkované responses
│       ├── PaginationParameters.cs # Sdílené parametry stránkování
│       └── QueryableExtensions.cs  # IQueryable<T>.ToPagedListAsync() extension
├── Exceptions/
│   ├── GlobalExceptionHandlerMiddleware.cs
│   └── ApiResultHelper.cs        # Obaluje handler volání, mapuje výjimky na HTTP responses
├── Extensions/
│   └── ValidationBehavior.cs     # FluentValidation pipeline behavior
```

```
├── Features/
│   └── Articles/              # Ukázková feature
│       ├── Commands/
│       │   ├── CreateArticle/     # 7 souborů
│       │   ├── UpdateArticle/
│       │   └── DeleteArticle/
│       └── Queries/
│           ├── GetArticles/
│           └── GetArticleById/
├── Middleware/
│   └── CorrelationIdMiddleware.cs
├── Program.cs                 # Minimální entry point
├── GlobalUsings.cs
├── appsettings.json
└── appsettings.Development.json
```

7-file struktura per operace

Každý command/query má přesně tyto soubory:

```
Features/{Feature}/Commands/{Operation}/
├── {Operation}Command.cs          # CQRS command — implements IQuery<TResponse>
├── {Operation}CommandHandler.cs   # Handler — implements IQueryHandler<TCommand, TResponse>, injectuje IDbContextFactory<PhysioBookContext>
├── {Operation}Request.cs          # HTTP request DTO (record)
├── {Operation}Response.cs         # HTTP response DTO (record, PascalCase properties)
├── {Operation}Validator.cs        # FluentValidation (AbstractValidator<TCommand>)
├── {Operation}Endpoint.cs         # MinimalAPI endpoint (static extension method)
└── {Operation}Mappings.cs         # Ruční mapping extension metody (Request→Command, Command→Entity, Entity→Response)
```

Pro queries je struktura analogická (Query místo Command).

CQRS Pattern — KRITICKÉ PRAVIDLO

Cortex.Mediator má POUZE `IQueryHandler<TRequest, TResponse>`. Neexistuje žádné `ICommandHandler`!

```
// SPRÁVNĚ — pro commands I queries:
public class CreateArticleCommandHandler : IQueryHandler<CreateArticleCommand, CreateArticleResponse>

// ŠPATNĚ — toto rozhraní NEEXISTUJE:
public class CreateArticleCommandHandler : ICommandHandler<CreateArticleCommand, CreateArticleResponse>
```

Command/Query třída implementuje `IQuery<TResponse>`:

```
public class CreateArticleCommand : IQuery<CreateArticleResponse>
{
    public required string Title { get; set; }
    public required string Content { get; set; }
    public string? UserId { get; set; }
}
```

GlobalUsings:

```
global using Cortex.Mediator;
global using Cortex.Mediator.Queries;
global using Cortex.Mediator.DependencyInjection;
global using FluentValidation;
global using Microsoft.EntityFrameworkCore;
```

Registrace mediatoru:

```
builder.Services.AddCortexMediator(builder.Configuration, new[] {
typeof(Program) });
builder.Services.AddTransient(typeof(IQueryPipelineBehavior<,>),
typeof(ValidationBehavior<,>));
```

Volání z endpointu:

```
var response = await mediator.SendAsync<CreateArticleCommand,
CreateArticleResponse>(command, ct);
```

JSON Serializace

Backend automaticky konvertuje PascalCase → camelCase:

Vrstva	Konvence	Příklad
C# DTOs	PascalCase	Title, CreatedAt
JSON API	camelCase	title, createdAt
TypeScript	camelCase	response.title

Autentizace — ASP.NET Identity + JWT

- ASP.NET Identity pro správu uživatelů (registrace, login, role)
- JWT Bearer tokeny pro API autorizaci
- Identity tabulky ve stejné PostgreSQL databázi
- Žádná externí závislost (žádný Keycloak, Auth0 apod.)

Role: Admin
Policies: Visitor (kdokoliv), Admin (Therapist)
Auth endpointy: /api/auth/register, /api/auth/login, /api/auth/refresh-token

Databáze — Entity pattern

Entity + EF konfigurace v jednom souboru. Handlery přistupují k datům přímo přes IDbContextFactory<PhysioBookContext> — žádný Repository pattern, žádný RepositoryBase<T>. EF Core DbContext je repository i Unit of Work — nepřidávej další abstrakční vrstvu.

Endpoint pattern

Endpointy se registrují centrálně v EndpointRegistration.cs (NIKDY v Program.cs).
Endpoint je static extension method na IEndpointRouteBuilder.
Každý endpoint volá ApiResultHelper.ExecuteWithErrorHandling pro konzistentní error handling.

Error handling

ApiResultHelper mapuje výjimky:
- ValidationException → 400
- KeyNotFoundException → 404

- UnauthorizedAccessException → 401
- InvalidOperationException → 400
- Exception → 500

Formát: ProblemDetails (RFC 7807).

Mapping pattern

Ruční statické extension metody — žádný AutoMapper:
- Request→Command (přidá server-side data jako UserId z JWT)
- Command→Entity
- Entity→Response

Frontend struktura

```
physiobook-frontend/
├── src/
│   ├── App.tsx                  # Routing (react-router-dom v6, lazy loading)
│   ├── main.tsx
│   ├── contexts/ (AuthContext.tsx, ToastContext.tsx)
│   ├── features/ (per feature: service, hooks, types)
│   ├── shared/ (BaseApiService.ts, ApiResponse.ts)
│   ├── components/ (Header, Footer, ProtectedRoute)
│   └── pages/
├── vite.config.ts, tailwind.config.js, tsconfig.json, package.json, Dockerfile
```

BaseApiService automaticky přidává JWT Bearer token do Authorization headeru.

Docker — docker-compose.yml

Všechny služby v kontejnerech, jeden příkaz spustí vše:
- postgres (PostgreSQL 15, port 5432)
- backend (ASP.NET Core 8, port 5050)
- frontend (React + Vite + Nginx, port 5173)

Testy

xUnit + FluentAssertions (NIKDY Verify snapshot testing) + Moq pro mocking.
Test pattern: {Feature}ValidatorTests, {Feature}HandlerTests.

Program.cs — minimální

```
var builder = WebApplication.CreateBuilder(args);
builder.ConfigureServices();   // → ServiceRegistrations.cs
var app = builder.Build();
app.ConfigureApp();            // → AppConfiguration.cs → EndpointRegistration
await DatabaseConfiguration.InitializeAsync(app.Services);
app.Run();
```

Požadovaný výstup

Vygeneruj kompletní scaffold se všemi soubory. Zahrň:
1. Celou backend strukturu včetně kompletní Articles CRUD feature (všech 5 operací × 7 souborů)
2. Auth endpointy (register, login, refresh)
3. Frontend s Articles stránkami (CRUD) a auth flow

4. Docker setup (Dockerfile pro backend, Dockerfile pro frontend, docker-compose.yml)
5. Testovací projekt s testy pro Articles validátory
6. Všechny konfigurační soubory (appsettings, .csproj, package.json, vite.config, tailwind.config)

Kód musí být funkční a připravený ke spuštění přes docker-compose up.

Začnu generovat celý projekt jako skutečné soubory a poskládám ho tak, aby šel rovnou otevřít a spustit přes Docker. Nejdřív připravím backend scaffold s Articles CRUD a auth, pak frontend, testy a nakonec zkontroluji strukturu a zabalím ti to jako artefakt.

Backend scaffold už skládám podle té 7-souborové struktury na operaci. Vedle toho řeším i praktickou spustitelnost: protože tu nemám .NET runtime ani Docker, nemohu projekt lokálně zkompilovat, takže dávám důraz na konzistentní wiring, konfiguraci a startup flow, aby byl scaffold co nejblíž „drop-in“ použitelnému stavu.

Frontend část teď doplňuji o hotové stránky, contexty a API vrstvu. Už vidím jeden detail, který jsem rovnou opravil v návrhu: refresh seznamu článků po smazání musí být oddělený od samotného search termu, aby se do dotazu neposílaly nesmyslné hodnoty.

Received app response

Received app response

Received app response

Received app response

Received app response

Received app response

Thought for 16m 48s

Hotovo.

Stáhnout:
PhysioBook scaffold ZIP
PhysioBook project folder

Obsahuje:

- backend ASP.NET Core 8 Minimal API
- Vertical Slice + CQRS scaffold
- kompletní Articles CRUD
- auth flow `register / login / refresh-token`
- React + Vite + TypeScript + Tailwind frontend
- Dockerfile pro backend i frontend + `docker-compose.yml`
- test projekt s validačními testy pro Articles
- solution file, README, appsettings, csproj, package.json, vite/tailwind configy

Backend jsem držel podle Cortex.Mediator 1.6.1 setupu a registrace, kterou má tahle verze zdokumentovanou. NuG +1

Jediná důležitá poznámka: v tomhle prostředí nemám .NET SDK ani Docker, takže jsem scaffold nemohl lokálně zbuildit a projet `docker-compose up`. Kód je poskládaný jako co nejblíže spustitelnému demo

lokálně zbuildit a projít docker-compose up. Řekni mi pokud by jsi chtěl upraveného demo základu, ale ber to poctivě jako scaffold bez runtime ověření.

Backend padá na DI chybě: "Cannot consume scoped service DbContextOptions from singleton IDbContextFactory<PhysioBookContext>". Oprav to a pošli mi Jen kusy kódu na opravu, ne celý projekt.

Podívám se přímo do vygenerovaného scaffoldu na registraci DbContextu a factory a pošlu ti jen přesné opravy souborů, které to lámou.

Thought for 35s

Jasně. Ta chyba vzniká tím, že máš současně `AddDbContext<PhysioBookContext>()` i `AddDbContextFactory<PhysioBookContext>()`, přičemž `IDbContextFactory` je defaultně singleton a snaží se sežrat scoped `DbContextOptions`.

Stačí upravit tyhle 2 soubory:

`Configurations/ServiceRegistrations.cs`

Nahraď tuhle část:

C#

```
services.AddDbContext<PhysioBookContext>(options =>
    options.UseNpgsql(configuration.GetConnectionString("DefaultConnection")));

services.AddDbContextFactory<PhysioBookContext>(options =>
    options.UseNpgsql(configuration.GetConnectionString("DefaultConnection")));
```

Tímhle:

C#

```
services.AddDbContextFactory<PhysioBookContext>(options =>
    options.UseNpgsql(configuration.GetConnectionString("DefaultConnection")));
```

`Configurations/DatabaseConfiguration.cs`

Nahraď celý soubor tímto:

C#

```
        const string adminEmail = "admin@physiobook.demo";
        const string adminPassword = "Admin123!";

        var adminUser = await userManager.Users.FirstOrDefaultAsync(x => x.Email ==
        if (adminUser is null)
        {
            adminUser = new ApplicationUser
            {
                Id = Guid.NewGuid().ToString(),
                UserName = adminEmail,
                Email = adminEmail,
                NormalizedEmail = adminEmail.ToUpperInvariant(),
                NormalizedUserName = adminEmail.ToUpperInvariant(),
                FullName = "Demo Therapist"
```

```
};

var createUserResult = await userManager.CreateAsync(adminUser, adminPa
if (!createUserResult.Succeeded)
```

Proč to funguje

Teď budeš používat jednotně `IDbContextFactory<PhysioBookContext>` všude tam, kde jsi ten pattern chtěl používat, a zmizí konflikt singleton factory vs scoped options.

Jestli chceš, hned ti pošlu i druhý malý fix, který budeš pravděpodobně potřebovat kolem `AddDbContextCheck<PhysioBookContext>()`, pokud by po téhle opravě začal zlobit health check.

Když přidám článek jako DemoTherapist, tak mi to napíše tuhle chybovou hlášku na FrontEndu VUI: "One or more validation errors occurred.
error"

Projdu backend validaci a mapování CreateArticle flow přímo v scaffoldu a vytáhnu jen místo, které to pravděpodobně láme. Pak ti dám přesný patch bez balastu.

Thought for 1m 8s

Jo — backend ti u validace vrací správný `ValidationProblem`, ale frontend z něj vytahuje jen `title/detail`, takže místo konkrétní chyby zobrazí jen generické:

> One or more validation errors occurred.

Oprav jen tyhle kusy:

`physiobook-frontend/src/shared/BaseApiService.ts`

Nahraď celý soubor tímto:

TypeScript

```
      problem = text ? (JSON.parse(text) as ProblemDetailsResponse) : undefined;
    } catch {
      problem = undefined;
    }

    throw new ApiError(this.formatProblem(problem), response.status, problem);
  }

  if (response.status === 204) {
    return undefined as T;
  }

  return (await response.json()) as T;
}

protected get<T>(path: string): Promise<T> {
  return this.request<T>(path, "GET");
}
```

`physiobook-frontend/src/pages/ArticleCreatePage.tsx`

Do `textarea` **přidej** `minLength={10}`:

TypeScript

```
<textarea
  className="min-h-64 w-full rounded-xl border border-slate-300 px-4 py-3 outline-none
  value={content}
  onChange={(event) => setContent(event.target.value)}
  required
  minLength={10}
/>
```

`physiobook-frontend/src/pages/ArticleEditPage.tsx`

Do `textarea` **přidej taky** `minLength={10}` **:**

TypeScript

```
<textarea
  className="min-h-64 w-full rounded-xl border border-slate-300 px-4 py-3 outline-none
  value={content}
  onChange={(event) => setContent(event.target.value)}
  required
  minLength={10}
/>
```

Pak místo generické hlášky uvidíš třeba:

```
content: 'Content' must be at least 10 characters. You entered 4 characters.
```

A zároveň tě browser nepustí odeslat formulář s kratším obsahem.

Na stránce http://localhost:5173/articles se nezobrazuji články

Podívám se přímo do scaffold souborů na list endpoint a frontend fetch flow. Nejčastěji to bývá buď špatná base URL, nebo mismatch mezi route a shape response.

Thought for 1m 14s

Nejspíš máš kombinaci **dvou věcí**:

- frontend při běhu přes **Vite dev server** posílá `GET /api/articles` na `localhost:5173`, ale ve `vite.config.ts` **nemáš proxy** na backend
- `useArticles()` chybu **tiše spolyká**, takže pak jen vidíš prázdný seznam

To přesně odpovídá chování „na `/articles` se nic nezobrazuje".

Oprav tyhle kusy:

`physiobook-frontend/vite.config.ts`

Nahraď tímto:

TypeScript

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";

export default defineConfig({
  plugins: [react()],
  server: {
    host: "0.0.0.0",
    port: 5173,
    proxy: {
      "/api": {
        target: "http://localhost:5050",
        changeOrigin: true
      },
      "/health": {
        target: "http://localhost:5050",
        changeOrigin: true
      }
    }
  }
```

physiobook-frontend/src/features/articles/articleHooks.ts

Nahraď celý soubor tímto:

TypeScript

```
        setItems([]);
        setError(err.message);
      }
    })
    .finally(() => {
      if (!isCancelled) {
        setIsLoading(false);
      }
    });

  return () => {
    isCancelled = true;
  };
}, [searchTerm, reloadToken]);

return { items, isLoading, error };
}

export function useArticle(id?: string) {
```

physiobook-frontend/src/pages/ArticlesListPage.tsx

Změň tento řádek:

TypeScript

```
const { items, isLoading } = useArticles(searchTerm, refreshKey);
```

na:

TypeScript

```
const { items, isLoading, error } = useArticles(searchTerm, refreshKey);
```

A přidej mezi loading a seznam tento blok:

TypeScript

```
{isLoading ? (
  <div className="rounded-3xl bg-white p-6 shadow-sm">Loading articles…</div>
) : error ? (
  <div className="rounded-3xl bg-red-50 p-6 text-sm text-red-700 shadow-sm">
    Failed to load articles: {error}
  </div>
) : (
```

Co je příčina

Když jedeš frontend přes `npm run dev`, `BaseApiService` používá:

TypeScript

```
"/api"
```

ale bez Vite proxy to jde na:

```
http://localhost:5173/api/articles
```

místo na backend:

```
http://localhost:5050/api/articles
```

Takže request spadne a hook to dosud vůbec nezobrazil.

Jestli chceš, pošlu ti hned ještě druhý malý patch, aby frontend fungoval korektně **jak ve Vite dev režimu, tak i přes Docker/Nginx build** bez přepínání konfigurace.

Failed to load articles: Required parameter "int PageNumber" was not provided from query string.

To už ukazuje přesně na binding problému v list endpointu. Opravím jen místo, kde se query DTO váže z URL, aby `pageNumber` a `pageSize` měly bezpečné defaulty i když je frontend neposílá.

Thought for 19s

To je backend binding bug v `GetArticlesRequest`.

`[AsParameters]` u Minimal API bere non-nullable `int` jako povinný parametr z query stringu. Default hodnota `= 1` ho v tomhle případě nespasí. Proto endpoint spadne, když frontend neposílá `pageNumber` a `pageSize`.

Oprav jen tyhle 2 soubory:

`PhysioBook.Api/Features/Articles/Queries/GetArticles/GetArticlesRequest.cs`

Nahraď tímto:

C#

```csharp
namespace PhysioBook.Api.Features.Articles.Queries.GetArticles;

public sealed record GetArticlesRequest
{
    public int? PageNumber { get; init; }
    public int? PageSize { get; init; }
    public string? SearchTerm { get; init; }
}
```

`PhysioBook.Api/Features/Articles/Queries/GetArticles/GetArticlesMappings.cs`

Nahraď metodu `ToQuery` tímto:

```csharp
public static GetArticlesQuery ToQuery(this GetArticlesRequest request)
    => new()
    {
        PageNumber = request.PageNumber ?? 1,
        PageSize = request.PageSize ?? 10,
        SearchTerm = request.SearchTerm
    };
```

Tím dosáhneš toho, že:

- `/api/articles` bude fungovat bez query parametrů
- `/api/articles?pageNumber=2&pageSize=20` bude fungovat taky
- validace přes FluentValidation zůstane zachovaná

Pro jistotu pak backend restartuj.

Kdybys chtěl, pošlu ti ještě malý frontend patch, aby Articles list rovnou explicitně posílal `pageNumber=1&pageSize=10` a bylo to konzistentní i do budoucna.